

内存管理： 虚拟内存与页面置换

理解操作系统如何让每个进程以为独占了全部内存——以及当物理内存不够时如何优雅地"换页"。

🕒 预计 45 分钟

📚 先修：进程与线程、C 指针

★ ★ 中等难度

第1章 · 虚拟内存

1.1 虚拟内存

概念

为什么需要

实现原理

1.2 页面置换

1.3 TLB

关键公式

$EAT = (1 - p) \times T_{\text{mem}} + p \times T_{\text{disk}}$

虚拟内存 (Virtual Memory)

操作系统为每个进程创建的**独立、连续**的地址空间抽象。进程看到的"地址"（虚拟地址）不是真实的物理内存地址——MMU 负责翻译。

关键洞察：**地址空间 ≠ 物理内存**。32 位系统每个进程"看到" 4GB，但物理内存可能只有 512MB。

地址翻译

物理地址 = 页表[虚拟页号] × 页大小 + 页内偏移

虚拟页号 (VPN) = 虚拟地址 / 页大小 | 页内偏移 = 虚拟地址 % 页大小 | 典型页大小 = 4KB

要点

32位地址 + 4KB页 → 20位 VPN (2²⁰=1M 个页表项)。如果每个 PTE 4 字节，单级页表就是 **4MB**——每个进程都要一份！所以需要多级页表。

第1章 · 虚拟内存

▸ 1.1 虚拟内存

概念

▸ 为什么需要

实现原理

1.2 页面置换

1.3 TLB

没有虚拟内存会怎样？三个致命问题

✗ 无隔离：进程 A 的野指针可以覆写进程 B 的数据，甚至内核内存

✗ 必须全量加载：游戏 60GB，你的 16GB 电脑根本跑不起来

✗ 外部碎片：频繁分配释放后，空闲空间被切成小块，无法分配大块连续内存

✅ 虚拟内存解决全部三个问题：隔离 → 独立地址空间 | 部分加载 → 按需调页 | 碎片 → 物理内存可非连续分配

实际案例：游戏的 60GB 纹理

《最终幻想》安装 60GB。启动时操作系统只加载当前场景需要的纹理页面到 RAM（约 2-3GB），其余留在 SSD。当你走到新区域时触发缺页中断，按需加载。整个过程对游戏透明——它"以为"自己有 60GB 连续内存。

第1章 · 虚拟内存

▸ 1.1 虚拟内存

概念

为什么需要

▸ 实现原理

1.2 页面置换

1.3 TLB

📄 页表大小

$$\text{Size} = 2^{32} / 2^{12} \times 4 = 4\text{MB}$$

1 CPU 发出虚拟地址

进程执行 `mov eax, [vaddr]`。vaddr 被拆分为 VPN + offset。

2 MMU 查 TLB

TLB 是页表的硬件缓存（~64-1024 项）。命中 → 直接拿到物理页框号 PFN（~1 cycle）。

3 TLB miss → 页表遍历 (Page Walk)

MMU 从 CR3 寄存器取页表基址，逐级遍历（x86-64 用 4 级页表：PML4→PDPT→PD→PT）。

4 PTE 的 Present 位 = 0 → Page Fault

CPU 陷入内核，OS 的 page fault handler 从磁盘换入页面、更新页表、重试指令。

💡 要点

TLB 覆盖率 = TLB 条目数 × 页大小。64 条 × 4KB = 256KB，但程序的工作集通常远超此值。这就是 TLB miss 不可避免的原因。

第2章 · 页面置换

1.1 虚拟内存

▸ 1.2 页面置换

▸ 概念

算法对比

Belady 异常

1.3 TLB

页面置换 (Page Replacement)

当物理内存已满、需要加载新页面时，操作系统必须**选择一个 victim page** 驱逐到磁盘 (swap out)，为新页面腾出空间。

核心目标：**最小化缺页率** → 最大化系统吞吐量。选错 victim 会导致刚换出的页立刻又被访问 (thrashing)。

为什么不能随便选？

选正在用的页面 → 刚换出又换入，磁盘 I/O 飙升

理想情况：选 未来最长时间不会被访问 的页面 → Belady 最优算法（理论最优，不可实现）